Modelos de Casos de Uso

Introducción

Los casos de uso son una técnica para analizar y especificar el comportamiento de un sistema o de una parte de este describiendo qué hace el sistema, pero no cómo lo hace.
Es una técnica de descubrimiento de requerimientos que se introdujo por primera vez en el método Objectory (Jacobson et al., 1993) y forma parte del estándar UML (Unified Model Language).
Si bien en un principio el modelado con casos de uso fue presentado para usarlo bajo la metodología de desarrollo orientada a objetos, posteriormente se lo utilizo en otras metodologías, tanto como parte del ciclo de vida de desarrollo del sistema como en el modelado ágil.

Es importante destacar que los CU poco tienen que ver con entender a un sistema como un conjunto de objetos que interactúan, que es la premisa básica del análisis orientado a objetos "clásico". En este sentido, el éxito de los casos de uso no hace más que dar la razón al análisis estructurado, que propone que la mejor forma de empezar a entender un sistema es a partir de los servicios o funciones que ofrece a su entorno, independientemente de los objetos que interactúan dentro del sistema para proveerlos.

Los casos de uso capturan de manera natural y especifican los requisitos del sistema a desarrollar. Como sabemos, todos los sistemas ofrecen a aquellos que lo usan una serie de servicios o funcionalidades, en este contexto podemos decir entonces que un caso de uso es una forma de expresar como "algo o alguien" externo al sistema interactúa con el mismo usando dichas funcionalidades y obteniendo un resultado de interés para sí mismo. Cuando decimos "algo o alguien" estamos haciendo referencia a que los sistemas no solo son usados por personas, sino también por otros sistemas de hardware y/o software. En cualquier caso, llamaremos "actor" a ese "algo o alguien" que interactúa con el sistema**.**

Actores

¿Qué es un actor?

Un actor es una agrupación uniforme de personas, sistemas, dispositivos o maquinas que interactúan con el sistema que estamos analizando de la misma forma, "algo o alguien" que utiliza el sistema

- ➔ "algo" que usa el sistema puede ser otro sistema software o un sistema hardware
- ➔ "alguien" que usa el sistema es representado por un rol particular que puede asumir una persona o un conjunto de personas al interactuar con el sistema que se está construyendo.

Los actores siempre son externos al sistema, por lo tanto, al identificarlos se comienza a definir el límite y el alcance de este.

Supongamos que debemos analizar el Sistema de Préstamos de la Biblioteca Leopoldo Marechal de la UNLaM quien recibe los pedidos de los alumnos para retirar un libro de la biblioteca, si pueden hacer las mismas cosas con el sistema, son considerados un único actor: Auxiliar de Biblioteca.

¿El actor, es el usuario?

Existe una diferencia entre el actor y el usuario, el actor representa un rol y el usuario es una persona o grupo de personas que cuando usa el sistema asume un rol, por lo tanto, puede ocurrir que una misma persona (usuario) en distintos momentos asuma roles diferentes y utilice el sistema de acuerdo al rol asumido. Por ejemplo, una persona que trabaja en un banco podría ser un *Ejecutivo de Cuentas* y asumiría ese rol para interactuar con el sistema cuando atiende a un *Cliente*, pero a su vez, si esta persona tiene sus cuentas personales en ese banco estaría desempeñando también el rol de *Cliente*.
El rol varía de acuerdo con el posicionamiento que se tiene frente al sistema en distintos momentos de acuerdo con lo que necesite utilizar el sistema un mismo usuario asumirá roles distintos.

Tipos de actores

Como se ha mencionado, los actores no siempre representan personas, también puede representar sistemas software o sistemas hardware. Por ejemplo, si se está construyendo un Sistema de Ventas que además de registrar las Ventas genere los asientos contables para ser procesados por el Sistema de contabilidad, éste último sería un actor ya que es externo al sistema de Ventas, pero interactúa con él. Por otro lado, pensemos por ejemplo en un robot, si el sistema que se está construyendo contiene las rutinas necesarias para controlar sus movimientos, el Robot sería el actor que interactúa con el mismo. Otros ejemplos de este tipo de actores que representan sistemas hardware podrían ser un sensor, una impresora, un lector de código de barras, un pos, etc.

¿Cómo se representan los actores?

Según el estándar UML 2.0, un actor se representa con un hombre de palo (stick man) con el nombre del rol que está cumpliendo debajo, o el nombre del sistema, si es que el actor está representando un sistema software o un sistema hardware.



Figura 1: Actor

Generalización de actores

¿Qué pasa si hay dos actores que hacen lo mismo? ¿Puede ocurrir?

Muchas veces puede ocurrir que un actor inicie todos los casos de uso que inicia otro actor y algunos más, por ejemplo, en un sistema de Ventas, el Supervisor de Ventas puede hacer todo lo que hace el Empleado de Ventas, pero además puede autorizar las Ventas, en este caso se puede decir que el supervisor de Ventas hereda al empleado de Ventas, o que el empleado de Ventas generaliza al supervisor de Ventas, de esta forma toda la funcionalidad que está habilitada para el empleado de Ventas también lo está para el supervisor. Usando la generalización de actores se evita la redundancia y se simplifica la especificación y el diagrama de casos de uso al no tener que trazar tantas relaciones entre actores y casos de uso.

Ejemplo:



ación de actores

Como se puede ver en siguiente ejemplo, la herencia de actores se representa mediante una relación de generalización, y la relación entre los actores y los CU mediante una asociación.

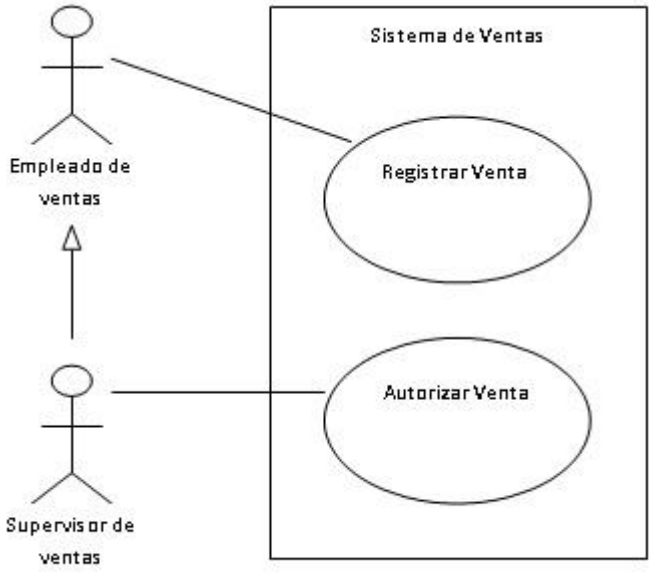


Figura 3: Ejemplo de generalización de actores

Casos de Uso

¿Qué es un Caso de Uso?

Un caso de uso se define como un conjunto de acciones realizadas por el sistema que dan lugar a un resultado observable.

Es iniciado por un único actor a la vez y representa *una funcionalidad completa* del sistema que se describe mediante la secuencia de interacciones entre el sistema y el actor que inicia el caso de uso. La ejecución de un caso de uso debe dejar un resultado observable y de interés para el actor que lo inició.
Cada caso de uso representa una *tarea completa concreta* que implica interacción externa con un sistema. En su forma más simple, un caso de uso se muestra como una elipse, con los actores que intervienen en el caso de uso representados como figuras humanas unidas por una asociación.

Los casos de uso se utilizan para especificar los *requisitos funcionales del sistema*, en cada caso de uso, la interacción entre el actor y el sistema involucra todos los pasos necesarios para llevar a cabo una funcionalidad dada describiendo *qué* hace el sistema, pero sin especificar *como* lo hace. Solo se pueden ver las entradas y las salidas del sistema, pero nunca los procesos.
Cuando se describe un caso de uso se debe especificar el escenario principal, es decir, la secuencia de interacciones que sigue el curso normal de los acontecimientos, y las alternativas. Las alternativas son errores o eventos excepcionales que conforman otro escenario dentro de la descripción del caso de uso distinto al escenario principal deseado para la obtención el objetivo de la ejecución del CU.

Escenarios

Un escenario es una secuencia específica de acciones que describe un comportamiento. La descripción de un caso de uso define uno o más escenarios, el escenario principal, que describe el curso normal de los acontecimientos y uno o más escenarios secundarios donde se describen las alternativas. Rumbaugh, Booch y Jacobson en su libro UML 2.0 (Rumbaugh, Jacobson, & Booch, El Lenguaje Unificado de Modelado UML 2.0, 2da ed. 2006) plantean que los escenarios son a los casos de uso lo que las instancias a las clases, es decir, un escenario es básicamente una instancia de un caso de uso.

¿Cómo se representan los casos de uso?

Según el estándar UML 2.0, un caso de uso se representa como una elipse que lleva el nombre del caso de uso en su interior[1]. El nombre de un caso de uso se expresa desde el punto de vista del actor que lo inicia y está compuesto por un *verbo en infinitivo seguido del objeto o entidad del sistema que es afectada por el mismo*, por ejemplo, Registrar *Venta*, Anular *pedido*, *Autorizar Venta*, etc.



Figura 4: Caso de uso

Características de los casos de uso

Los casos de uso tienen las siguientes características:

- Están expresados desde el punto de vista del actor y no del sistema, el punto de vista del actor representa una vista externa del sistema, por ejemplo, el caso de uso Ingresar pedido iniciado por el actor Empleado de Ventas, permite que el empleado de Ventas ingrese un pedido en el sistema de Ventas (visto desde su punto de vista), pero si se ve desde el punto de vista del sistema, el mismo caso de uso podría llamarse Recibir información de pedido. Debe destacarse que distintos casos de uso pueden cumplir un mismo objetivo, aunque se llamen distinto y tenga actores “opuestos” Ej. Si un cliente realiza la compra de una entrada para ver una película en un Complejo de cines, si el cliente realiza la compra desde una terminal de auto gestión, desde una App o des una aplicación web entonces el Cliente es el actor y el caso de uso se llamará Comprar Entrada a Cine, si en cambio el cliente concurre a una Ventanilla del Complejo para adquirir su entrada, el actor es el Vendedor (quien opera el sistema) y el caso de

[1] Si bien están basadas en el estándar UML, algunas herramientas de modelado colocan el nombre del caso de uso debajo del ovalo en lugar de en su interior, por lo tanto, las variaciones en la notación dependerán de la herramienta utilizada a la hora de modelar.

uso se llamará Vender entrada de Cine. Esta diferencia debe quedar muy cara al realizar el modelado del sistema.

- Se documentan con texto informal, en primera instancia, el documento que detalla la especificación de los casos de uso va a estar dirigido al cliente, por lo tanto, se debe documentar utilizando un lenguaje natural para que el cliente lo entienda.

- Describen tanto lo que hace el sistema como lo que hace el actor cuando interactúa con él.

- Son iniciados por un único actor a la vez, un mismo caso de uso puede ser iniciado por más de un actor en momentos diferentes, pero no puede haber casos de uso que no sean iniciados por ningún actor.

- Están acotados a una sola funcionalidad completa del sistema, una funcionalidad completa es un caso de uso si se debe indicar explícitamente al sistema que se quiere acceder a dicha funcionalidad. Por ejemplo, si se quiere registrar una Venta se debe acceder a la funcionalidad Registrar Venta, en cambio, si solo se quieren ingresar los artículos vendidos para una Venta en particular no debemos indicárselo explícitamente al sistema, simplemente esa es una funcionalidad que forma parte de una funcionalidad mayor que es Registrar Venta, por lo tanto, no sería un caso de uso.

- Los casos de uso deben dejarle un beneficio o resultado de interés al actor que lo inició, por ejemplo, el beneficio que obtiene el Empleado de Ventas al ejecutar el caso de uso Registrar Venta, es que la Venta queda registrada.

Modelo de Casos de Uso

El modelo de casos de uso (MCU) presenta al sistema desde la perspectiva del actor y facilita la comunicación entre el cliente y el equipo de desarrollo para llegar a un acuerdo sobre los requisitos, condiciones y posibilidades que debe cumplir el sistema. Luego de la primera revisión del cliente, este modelo se irá refinando hasta llegar a su versión final que servirá como acuerdo o contrato donde se especifican todas las funcionalidades que el sistema debe cumplir y el equipo de desarrollo se compromete a desarrollar.

El MCU está compuesto por:

- El diagrama de casos de uso.

- La descripción de los casos de uso. (Descripción General de la Especificación del CU)

Diagrama de Casos de Uso

El diagrama de casos de uso está compuesto por un rectángulo que representa y delimita el sistema, sobre la línea superior se detalla el nombre del mismo, y dentro de los límites del rectángulo se incluyen los casos de uso, las relaciones entre actores y casos de uso y relaciones entre casos de uso.
Los actores son externos al sistema, por lo tanto, siempre se dibujan fuera del rectángulo y se representan con el (stick man) y el nombre que debe indicar qué rol está cumpliendo ese actor. Dentro del rectángulo *estará lo que hace el sistema,* todas sus funcionalidades representadas por casos de uso, cada caso de uso se va a representar con una elipse/óvalo que lleva su nombre en el interior, recordando que el nombre debe comenzar con un verbo en infinitivo. En el diagrama debe visualizar claramente cuáles son los casos de uso iniciados por cada actor mediante la asociación que los une.
Si existieran, también se deben graficar las relaciones de generalización entre actores y las relaciones de inclusión y extensión entre casos de uso, *estas últimas al estar relacionando funcionalidades, se dibujan siempre dentro del rectángulo que representa al sistema*. Las relaciones entre casos de uso se explicarán más adelante dentro de este capítulo.

Ejemplo:

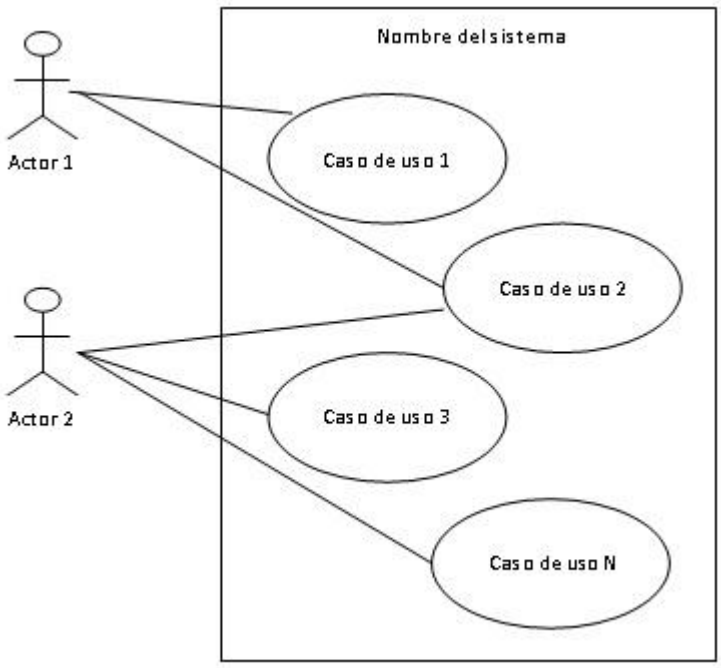


Figura 5: Diagrama de casos de uso

Descripción de los Casos de Uso

Luego de realizar el diagrama se debe hacer una descripción completa de cada uno de los casos de uso incluidos en el mismo.
Se trata de una descripción de carácter general donde se debe enunciar de manera clara y precisa el objetivo del caso de uso, que se espera de su ejecución, el beneficio que deja el caso de uso para el actor y quién es el actor involucrado. Esta descripción tiene una relación directa con el nombre del caso de uso.

Aplicación Práctica del Modelo de Casos de Uso

Trabajaremos con el ejercicio de la Biblioteca Leopoldo Marechal de la UNLaM focalizándonos en los préstamos que se realizan a los alumnos de dicho Establecimiento.

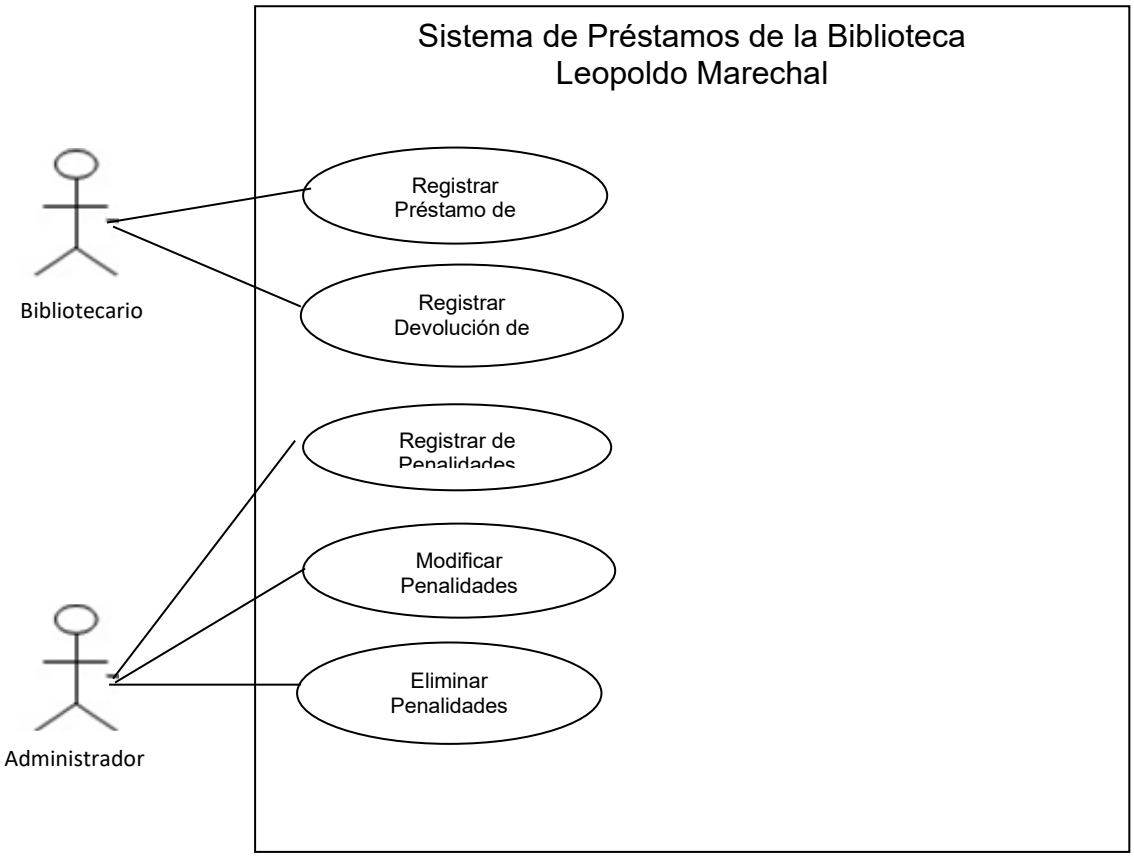
Identificamos los siguientes actores:

- Bibliotecario
- Administrador

Identificamos los siguientes Casos de Uso:

- Registrar Préstamo de Libros
- Registrar Devolución de Libros
- Registrar de Penalidades
- Modificar de Penalidades
- Eliminar de Penalidades

Diagrama de Casos de Uso



Descripción General de los CU

Registrar Préstamo de Libros: El bibliotecario recibe el DNI del alumno que solicita el préstamo de un libro, ingresa los datos de identificación del mismo. El caso de uso se comunica con el Sistema de Alumnos y con el Sistema Pérgamo, para conocer la condición del alumno, y del ejemplar. Si el alumno está ok y hay disponibilidad del ejemplar se registra el préstamo y se actualiza el estado de disponibilidad del ejemplar.

Registrar Devolución de Libros: El bibliotecario recibe el ejemplar del libro, ingresa los datos de este y registra la devolución. En caso de no cumplir con los plazos de devolución o el ejemplar se encuentre en mal estado, se aplican penalidades informándolo al Sistema de Alumnos y al Sistema Pérgamo para cambiar el estado del ejemplar a disponible.

Registrar Penalidades: El administrador registra los distintos tipos de penalidades que pueden aplicarse al momento de la devolución de un libro. Si un libro fue devuelto en mal estado, dañado en general, fuera de término o si se extravió, le aplicaran al alumno responsable distintos tipos de penalidades.

Modificar Penalidades: El administrador realiza cambios a los distintos tipos de penalidades que pueden aplicarse al momento de la devolución de un libro, cambiando los plazos, los motivos, o la gravedad de estos.

Eliminar Penalidades: El administrador puede eliminar algún tipo de penalidad que puede aplicarse al momento de la devolución de un libro, por obsolescencia, cambio de criterio de sancionatorio, etc.

Metodología de trabajo para Identificación de Casos de Uso

Rumbaugh, Booch y Jacobson en su libro "*El proceso unificado de desarrollo de software*" (Rumbaugh, Jacobson, & Booch, El Proceso Unificado de Desarrollo de Software, 1ra ed. 2000) plantean una metodología de trabajo para realizar en forma correcta el MCU que consiste básicamente en encontrar los actores que van a interactuar con el sistema, encontrar los casos de uso para cada actor y describirlos primero brevemente y luego en forma detallada para completar el modelo. Identificar actores y casos de uso es una actividad muy importante ya que ayuda a delimitar el sistema y a definir su alcance.
Si bien esta actividad es responsabilidad del analista de sistemas, se requiere un grado de participación importante del cliente, ya que este último es quien proporciona los requerimientos y la información acerca de los distintos roles y requerimientos/funcionalidades del sistema que se transformaran en casos de uso.

El desarrollo del MCU es iterativo, el analista de sistemas junto a su equipo puede crear un modelo preliminar que luego se validará con el cliente quien podría sugerir

modificaciones o agregados de funcionalidades nuevas, de esta manera se va refinando el modelo hasta llegar a la versión definitiva que será aprobada por el cliente y sirve como contrato donde se especifican todas las funcionalidades que el sistema debe cumplir.

Permite representar distintos escenarios que piden presentarse al desarrollarse una funcionalidad completa del sistema a modelar.

Un escenario es una secuencia de acciones/pasos que permite describir la funcionalidad del caso de uso.

Escenario básico (Flujo Normal -> todo va bien)
Escenarios alternativos (Flujos Alternativos -> gestión de errores y/oexcepciones)

Aplicar esta metodología consiste en:

1) **Identificar actores**: antes de avanzar con los casos de uso se debe tratar de identificar a todos los actores que van a interactuar con el sistema. El analistade sistemas junto con el cliente tiene la tarea de identificar los usuarios y organizarlos según el rol que van a representar al interactuar con el sistema, esta tarea permite definir una lista de actores candidatos. En un primer momento es común encontrar varios actores que cumplan con el mismo rol, en estos casos habrá que revisar la lista de actores candidatos para finalmente encontrar el conjunto de actores adecuados y eventualmente definir generalizaciones en los casos necesarios. El paso siguiente será asignarle a cada actor un nombre que defina claramente el rol que está representando y describir brevemente para que utilizará el sistema, indicando necesidades y responsabilidades.

   Como resultado de este paso se obtiene un conjunto de actores actualizado, cada uno con su descripción, que pueden utilizarse como punto de partida para encontrar los casos de uso.

2) **Identificar los casos de uso para cada actor**: el analista de sistemas irá revisando los actores y proponiendo los casos de uso para cada uno, estos casos de uso propuestos en primera instancia son casos de uso candidatos yno todos llegarán a ser casos de uso por sí mismos. Recordemos que un caso de uso debe representar una funcionalidad completa del sistema y dejar un resultado de valor para el actor, revisar minuciosamente los casos de uso candidatos es una tarea importante ya que este análisis permite identificar las funcionalidades completas que representan casos de uso en sí mismos y las que son sólo parte de otro caso de uso. Una vez encontrados los casos de uso para cada actor se le debe asignar un nombre a cada uno, el nombre de un caso de uso siempre comienza con un verbo en infinitivo y debe reflejar

cual es el objetivo de la interacción entre el actor y el sistema visto desde el punto de vista del actor, por ejemplo, un caso de uso llamado *Registrar Venta* refleja claramente que el objetivo del *Empleado de Ventas* (actor) al interactuar con el sistema es que la Venta quede registrada en el mismo.

Como resultado de este paso, se obtiene un primer diagrama de casos de uso sujeto a evaluaciones y restructuraciones.

3) **Identificar nuevos casos de uso a partir de los existentes**: en este paso se debe verificar si el sistema está completo o completarlo buscando otras funcionalidades a partir de:

   a. *Variaciones significativas de los casos de uso existentes*: se pueden encontrar nuevos casos de uso a partir de las variantes de un caso deuso existente. Por ejemplo, si existieran muchas diferencias entre registrar la Venta a un cliente minorista y registrar la Venta a un cliente mayorista, entonces, del caso de uso *Registrar Venta* se podrían obtener los casos de uso *Registrar Venta a cliente minorista* y *Registrar Venta a cliente mayorista*, siempre y cuando las variaciones del caso de uso lo justifiquen.

   b. *Casos de uso opuestos*: se podrían encontrar nuevas funcionalidades para completar el sistema buscando casos de uso opuestos a los existentes, por ejemplo, si existe el caso de uso *Registrar Venta*, debería existir también el caso de uso *Anular Venta*.

   c. *Casos de uso que preceden a casos de uso existentes*: siempre hay que preguntarse qué sucede antes del caso de uso que se está analizando. Por ejemplo, si se está registrando una Venta a un cliente, el cliente ya tendría que estar registrado como tal en el sistema, por lo tanto, debería existir otro caso de uso llamado *Registrar Cliente*.

   d. *Casos de uso que suceden a los casos de uso existentes*: en este caso hay que preguntarse qué ocurre después del caso de uso que se está analizando. Por ejemplo, ¿qué ocurre después de *Registrar Venta*?, ésta es una buena forma de asegurarse que no se están dejando requerimientos sin identificar y permite descubrir partes de alguna nueva funcionalidad y nuevos casos de uso.

   Luego de analizar todos estos casos, se obtiene como resultado un segundo diagrama de casos de uso más completo donde se agregan los nuevos casosde uso encontrados.

4) **Describir los casos de uso con una Descripción General**: una vez identificados todos los casos de uso hay que empezar a documentarlos

haciendo la descripción con trazo grueso. Recordemos que ésta debe ser una breve descripción, realizada desde la perspectiva del actor, que especifique lo que hace el caso de uso indicando quien es el actor involucrado, la funcionalidad que se va a llevar a cabo y cuál es el resultado de valor que deja ese caso de uso para el actor.

Como resultado de este paso se obtiene el MCU compuesto por el diagrama de casos de uso completo y la descripción general de cada caso de uso.

Este MCU representa los requerimientos del cliente "convertidos" en casos de uso que especifican las funcionalidades que el sistema debe cumplir junto a susdescripciones y a los actores que van a interactuar con las mismas.

Llegado este punto, el cliente deberá ver el MCU para determinar si los casosde uso capturan todos sus requerimientos, si la descripción de los mismos es correcta, completa y comprensible y si identifica algún caso de uso que no proporcione ningún resultado de valor, en cuyo caso habría que reconsiderar la inclusión del mismo en el modelo.

Es muy importante que el analista de sistemas junto con su equipo revise bien este modelo antes de presentárselo al cliente ya que el mismo servirá como contrato donde se especifica todo lo que el sistema "garantiza que cumplirá" siel cliente decide aceptar y se avanza con el proyecto.

5) **Priorizar los casos de uso:** en este paso se deben definir prioridades y seleccionar los casos de uso de la primera iteración. Una vez documentados loscasos de uso con su Descripción General, es conveniente definir las prioridadesde estos, para determinar cuáles serán tenidos en cuenta en las primeras iteraciones del proceso de desarrollo y cuáles se pueden dejar para más adelante. Sabiendo que los casos de uso especifican los requisitos funcionales del sistema basados en los requerimientos del cliente, para priorizarlos se debe tener en cuenta la siguiente categorización:

   a. Requerimientos *imprescindibles o de prioridad alta*, son aquellos requerimientos que si no se implementan el sistema no funcionaría y no tendría sentido.

   b. Requerimientos *importantes o de prioridad media*, son aquellos requerimientos muy deseables, pero no imprescindibles y se pueden negociar con el cliente.

   c. Los requisitos *deseables o de prioridad baja*, son aquellos requerimientos posibles de implementar pero que podrían eliminarse. Solo se implementan si hay tiempo disponible.

Una vez categorizados los casos de uso, se seleccionan los que se van a desarrollar en la primera iteración, en general, se incluyen los de prioridad alta, se discuten los de prioridad media y se descartan los de prioridad baja cuyo costo sea alto.

Como resultado de este paso se obtienen los casos de uso priorizados para comenzar a planificar cuáles se van a desarrollar en cada iteración del proceso de desarrollo.

6) **Especificar Detalladamente los casos de uso:** una vez seleccionados los casos de uso que se van a implementar en la primera iteración, se realiza la descripción detallada de cada caso de uso describiendo el paso a paso de interacción entre el actor y el sistema, siempre realizada desde *el punto de vistadel actor*, y se divide en el flujo normal, que muestra el curso normal de los acontecimientos, el caso en “que todo sale bien” y el flujo alternativo que muestra las desviaciones del curso normal por posibles errores o excepciones. Es muy importante describir todas las alternativas o de lo contrario no se estaríaespecificando por completo el caso de uso, estas desviaciones del curso normal, errores o excepciones pueden ocurrir por varias razones, por ejemplo, el actor puede elegir entre diferentes caminos en el caso de uso, sólo uno de esos caminos (el que habitualmente se va a seguir) se debe describir en elcurso normal y los demás se describen como alternativas. Otro ejemplo de errores o excepciones se puede dar cuando el sistema detecta una entrada errónea por parte del actor, lo cual también se debe contemplar como una alternativa al curso normal.
En este paso se crea un prototipo de interfaz del usuario para cada caso de uso a describir que ilustre cómo el actor puede ejecutarlo y brinde una idea de cómo se verá el sistema. Teniendo una interfaz tentativa donde se visualicen los datos involucrados en la ejecución del caso de uso resulta más sencillo escribir

la descripción detallada de la interacción indicando las acciones del actor y las respuestas del sistema.

Como resultado de este paso se obtiene la descripción detallada de los casosde uso junto a sus prototipos de interfaz.

7) **Identificar relaciones entre casos de uso:** luego de describir los casos de usocon trazo fino, revisando estas descripciones, se pueden detectar secuenciasde pasos que se repiten *siempre* en distintos casos de uso. Para evitar este tipode redundancia, si no existiera, se debe crear un nuevo caso de uso que describa esta secuencia de pasos repetitiva y relacionarlo con los casos de uso que incluyen dicha secuencia a través de una relación *include*, de esta manera cada vez que se ejecute un caso de uso A que incluya al caso de uso B, también se ejecutará B. Ahora bien, si se encuentra una secuencia

de pasos que se repiten en distintos casos de uso, pero solo en algunas oportunidades, dicha secuencia sería *opcional* o alternativa dentro de la ejecución de un caso de uso dado, y en caso de ejecutarse, estaría extendiendo o añadiendo funcionalidad al caso de uso en cuestión. En estos casos, si no existiera, se crea un nuevo caso de uso que describa la secuencia opcional y se lo relaciona con los casos de uso extendidos a través de una relación *extend*. De esta manera un caso de uso A que extiende a un caso de uso B se puede ejecutar o no cada vez que se ejecute B, la llamada a la ejecución del caso de uso A se hará en algún punto del flujo alternativo dentro de la descripción detallada del caso de uso B, una vez finalizado el caso de uso A se continua con la secuencia descripta dentro del trazo fino del caso de uso B.
Así se puede ver claramente la diferencia entre las relaciones extend e include, a diferencia del caso anterior, si un caso de uso A incluye al caso de uso B, la llamada a la ejecución del caso de uso B se hará en algún punto dentro del flujonormal de la descripción detallada del caso de uso A ya que B se ejecutará siempre que se ejecute A.

Como resultado de este paso se obtiene el diagrama de casos de uso completo incluyendo las relaciones entre casos de uso.

En resumen, al aplicar esta metodología de trabajo se obtiene el MCU que captura losrequisitos del sistema y que está compuesto por el diagrama de casos de uso, la descripción general y la descripción detallada de cada caso de uso. Utilizando el proceso unificado de desarrollo de software como marco de trabajo, el MCU obtenido será el punto de partida para completar las etapas de análisis, diseño, implementación y prueba ya que los casos de uso van a dirigir el trabajo a lo largo de estas etapas iteración por iteración.

Relaciones entre Casos de Uso

Existen dos tipos de relaciones entre casos de uso:

- Relaciones de extensión (extend)
- Relaciones de inclusión (include)

Estas relaciones se podrán detectar una vez hecha la descripción detallada de todos los casos de uso. Tanto los extend como los include se deben **ejecutar simultáneamente con la ejecución de caso de uso base**.

Relaciones de extensión: en una relación de extensión, un caso de uso que extiende a otro le agrega funcionalidad al caso de uso extendido y se ejecutará de manera *opcional* cada vez que se ejecute éste. La funcionalidad descripta en el caso de uso *"que extiende"* representa una serie de pasos opcionales o una excepción dentro de la funcionalidad descripta en el caso de uso *"extendido"* que se ejecutará

sólo en algunas oportunidades. Por ejemplo, supongamos que estamos describiendo el caso de uso *Registrar Venta*, puede ocurrir que tengamos que vender a un cliente nuevo que no esté registrado en el sistema, en este caso aparece una excepción, la excepción consiste en interrumpir la ejecución de este caso de uso y pasar a la ejecución del caso de uso *Registrar cliente*, de esta manera podemos decir que el caso de uso *Registrar cliente* que se ejecuta de manera opcional ***extiende*** o agrega funcionalidad al caso de uso *Registrar Venta*.

Las relaciones de extensión entre casos de uso se representan en el diagrama con una relación de dependencia desde el caso de uso "*que extiende a"* hacia el caso de uso *"extendido"* acompañada por el estereotipo UML <<extends>>.

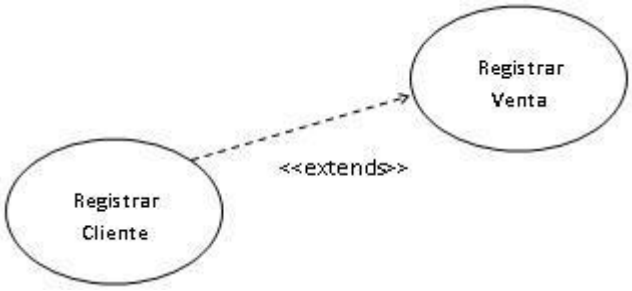


Figura 1: Relación de extensión

Además, las relaciones de extensión tienen las siguientes características:

- Representan una parte de la funcionalidad de un caso de uso que *no siempre*
  ocurre.
- La funcionalidad *"que extiende"* al caso de uso también es un caso de uso en símismo.
- Si bien es lo más común, no necesariamente provienen de un error o unaexcepción.

Diferencias entre una alternativa y una relación de extensión:

- En una relación de extensión, la funcionalidad que extiende a un caso de uso esun caso de uso en sí mismo mientras que una alternativa no lo es.
- Una alternativa es un error o excepción, mientras que una extensión puede noserlo.

Relaciones de inclusión: luego de realizar la descripción detallada de los casos de uso, puede ocurrir que se detecte alguna serie de pasos que se repite en dos o más

casos de uso. Si esta serie de pasos representa una funcionalidad completa, a partir de ella se crea un nuevo caso de uso que estará *"incluido"* en cada uno de los casos de uso que utilicen dicha funcionalidad y que se ejecutará *siempre* que éstos se ejecuten. Por ejemplo, la funcionalidad *Buscar producto* puede ser accedida por los casos de uso *Registrar pedido*, *Registrar Venta*, etc., se puede entonces, crear el caso de uso *Buscar producto* y relacionarlo a través de una relación de inclusión con los casos de uso que lo utilizan. De esta manera, *Buscar producto*, al ser el caso de uso ***incluido*** se ejecutará siempre que se ejecuten los casos de uso que lo incluyen.

Las relaciones de inclusión se representan en el diagrama con una relación de dependencia desde el caso de uso *"que incluye"* hacia el caso de uso *"incluido"* acompañada por el estereotipo UML <<include>>.

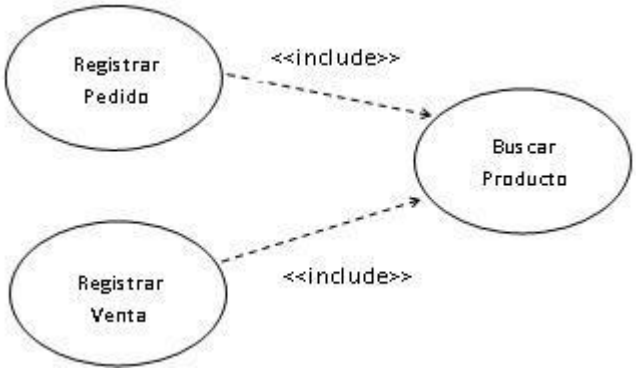


Figura 2: Relación de inclusión

Las características de las relaciones de inclusión son:

- Aparecen como una funcionalidad común luego de haber especificado varios casos de uso.
- Las funcionalidades incluidas son casos de uso en sí mismos.
- El caso de uso incluido se ejecuta *siempre* que el caso de uso que lo incluye se ejecute. Esta es la principal diferencia con las relaciones de extensión donde uncaso de uso que extiende a otro se puede ejecutar, o no, de manera opcional.